

Time Complexity

ในการวิเคราะห์ Time Complexity ของระบบ Emergency Call Dispatch Queue จะพิจารณาการทำงานหลัก ได้แก่ enqueue(), dequeue(), peek(), search() และ display()

Algorithm A: FIFO Event Queue

Algorithm A ใช้ ArrayDeque<Event> และทำงานตามหลัก FIFO

Operation	Method	Time Complexity
เพิ่ม Event	offer() / enqueue()	O(1)
นำ Event ออก	poll() / dequeue()	O(1)
ดู Event ด้านหน้า	peek()	O(1)
ค้นหา Event	search()	O(n)
แสดง Event ทั้งหมด	display()	O(n)

อธิบาย

1. Enqueue — O(1)

การเพิ่ม Event เข้าไปท้าย Queue สามารถทำได้ในเวลาคงที่

[E1, E2] → ENQUEUE E3 → [E1, E2, E3]

จึงมี Time Complexity เป็น O(1)

2. Dequeue — O(1)

การนำ Event ด้านหน้าออกสามารถทำได้โดยไม่ต้องเลื่อนสมาชิกทั้งหมด

[E1, E2, E3]

↓ DEQUEUE

[E2, E3]

จึงเป็น $O(1)$

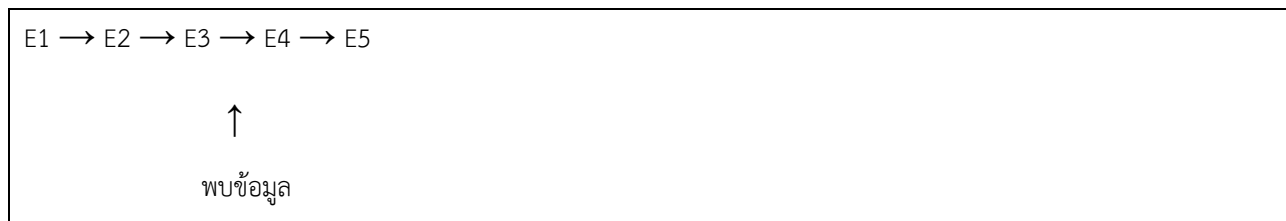
3. Peek — $O(1)$

การดู Event ที่อยู่ด้านหน้าไม่ต้องค้นหาข้อมูลทั้งหมด

จึงเป็น $O(1)$

4. Search — $O(n)$

การค้นหา Event เช่น E5 อาจต้องตรวจสอบตั้งแต่ Event ตัวแรกจนถึงตัวสุดท้าย



กรณีแย่ที่สุดต้องตรวจสอบทั้งหมด n รายการ จึงเป็น $O(n)$

5. Display — $O(n)$

การแสดงผล Event ทั้งหมดต้องเข้าถึงสมาชิกทุกตัว จึงเป็น $O(n)$

Algorithm B: Priority Event Queue

Algorithm B ใช้ `PriorityQueue<Event>` เพื่อจัดลำดับตาม Severity

Operation	Method	Time Complexity
เพิ่ม Event	<code>offer()</code>	$O(\log n)$
นำ Event ออก	<code>poll()</code>	$O(\log n)$
ดู Event ที่ Priority สูงสุด	<code>peek()</code>	$O(1)$
ค้นหา Event	<code>search()</code>	$O(n)$
แสดง Event ทั้งหมด	<code>display()</code>	$O(n)$

อธิบาย

1. Enqueue / Offer — $O(\log n)$

เมื่อเพิ่ม Event เข้า PriorityQueue ระบบต้องปรับโครงสร้าง Heap เพื่อรักษาลำดับ Priority

ENQUEUE E3



ตรวจสอบ Priority



ปรับตำแหน่งใน Heap

จึงมี Time Complexity เป็น $O(\log n)$

2. Dequeue / Poll — $O(\log n)$

เมื่อ poll() Event ที่มี Priority สูงสุดออก ระบบต้องปรับ Heap ใหม่

ตัวอย่าง

E3(Critical)



E5(Critical)

E4(High)

E1(Medium)

จึงมี Time Complexity เป็น $O(\log n)$

3. Peek — $O(1)$

peek() สามารถดู Event ที่มี Priority สูงสุดได้โดยไม่ต้องนำออกจาก Queue

จึงเป็น $O(1)$

4. Search — $O(n)$

การค้นหา Event เช่น E5 ไม่สามารถใช้ Priority เพื่อค้นหา Event โดยตรงได้ จึงอาจต้องตรวจสอบสมาชิกทั้งหมด

ดังนั้นเป็น $O(n)$

5. Display — $O(n)$

หากต้องการแสดง Event ทั้งหมด ต้องเข้าถึง Event ทุกตัว จึงเป็น $O(n)$

หมายเหตุ: การ iterate PriorityQueue ของ Java ไม่ได้หมายความว่าเราได้ลำดับ Priority ที่เรียงสมบูรณ์ ดังนั้นถ้าต้องการแสดงลำดับการประมวลผลจริง ควรใช้ poll() ทีละตัว

เปรียบเทียบ Time Complexity

Operation	FIFO ArrayDeque	Priority PriorityQueue
enqueue() / offer()	$O(1)$	$O(\log n)$
dequeue() / poll()	$O(1)$	$O(\log n)$
peek()	$O(1)$	$O(1)$
search()	$O(n)$	$O(n)$
display()	$O(n)$	$O(n)$

5.4 วิเคราะห์โดยรวม

FIFO Queue มีประสิทธิภาพสูงกว่าในด้านการเพิ่มและนำ Event ออกจาก Queue เนื่องจาก enqueue() และ dequeue() ใช้เวลา $O(1)$

ส่วน Priority Queue มีค่าใช้จ่ายมากกว่า เนื่องจากการ offer() และ poll() ต้องรักษาลำดับของ Heap จึงใช้เวลา $O(\log n)$

อย่างไรก็ตาม Priority Queue มีข้อได้เปรียบที่สำคัญสำหรับระบบ Emergency Call Dispatch คือสามารถเลือก Event ที่มี **Severity สูงที่สุด** ได้ทันทีด้วย peek() หรือ poll() ซึ่งเหมาะกับระบบฉุกเฉินที่ต้องให้ความสำคัญกับเหตุการณ์รุนแรงก่อน

Algorithm A: FIFO Event Queue มี `enqueue()` และ `dequeue()` เป็น $O(1)$ จึงเหมาะกับระบบที่ต้องการประมวลผล Event ตามลำดับเวลาและต้องการประสิทธิภาพสูง

Algorithm B: Priority Event Queue มี `offer()` และ `poll()` เป็น $O(\log n)$ และ `peek()` เป็น $O(1)$ เนื่องจากใช้ Heap ในการจัดลำดับ Priority แต่เหมาะกับระบบ Emergency Call Dispatch มากกว่า เพราะสามารถประมวลผลเหตุการณ์ที่มีความรุนแรงสูงก่อนเหตุการณ์ทั่วไปได้